
REQUIREMENTS NOT MET

N/A

PROBLEMS ENCOUNTERED

None encountered.

HOMEWORK EXERCISES

- i. What is the purpose of the Configuration Change Protection (CCP) register?

To prevent editing specific registers that are deemed critical to the processors operations from being accidentally written to. It disables signals from editing the register unless you write 0xD8 for IO registers or 0x9D for LPM/SPM first.

- ii. What would need to be done for the ATxmega128A1U to have a system clock frequency of 8 MHz?

You would have to use a prescaler of 4, through PSADIV[4:0] bits in the PSCTRL register, by writing (0x03<<2).

- iii. An internal 32 MHz clock is available in ATxmega128A1U it can also run much faster. Describe how the ATxmega128A1U may be configured to run clock frequencies of 64 MHz or even 128 MHz, without using any external clock sources? See section 37.2.14 of doc8385.

You would utilize the phase-locked loop hardware, which acts as a multiplier for the internal clock frequency, from 1-31x. (or divides the output by 2).

This behavior is controlled from the PLLCTRL register through PLLFAC[4:0], which takes a binary input that acts as a multiplier for the clock frequency.

PSEUDOCODE/FLOWCHARTS

BOTH SECTIONS (1 & 2) / (4MHz & 32MHz)

Include Files

Extern .s File

Int main(void){

Clock_init

Send signal to PORTC7

Set PORTC7 to output

While(1){

// Loop

}

}

PROGRAM CODE

BOTH SECTIONS (1 & 2) / (4 MHz & 32MHz)

```
/*
 * hw4.c
 *
 * Created: 7/24/2026 5:52:37 PM
 * Author : Evan Baesler
 */

#include <avr/io.h>

extern void clock_init(void);

int main(void)
{
    clock_init();
    PORTCFG.CLKEVOUT = PORTCFG.CLKOUT_PC7_gc;
    PORTC.DIRSET = (1<<7);

    while (1)
    {
        // Loop while reading CLK
    }
}

/* .s file is below in Appendix per template specifications */
```

APPENDIX

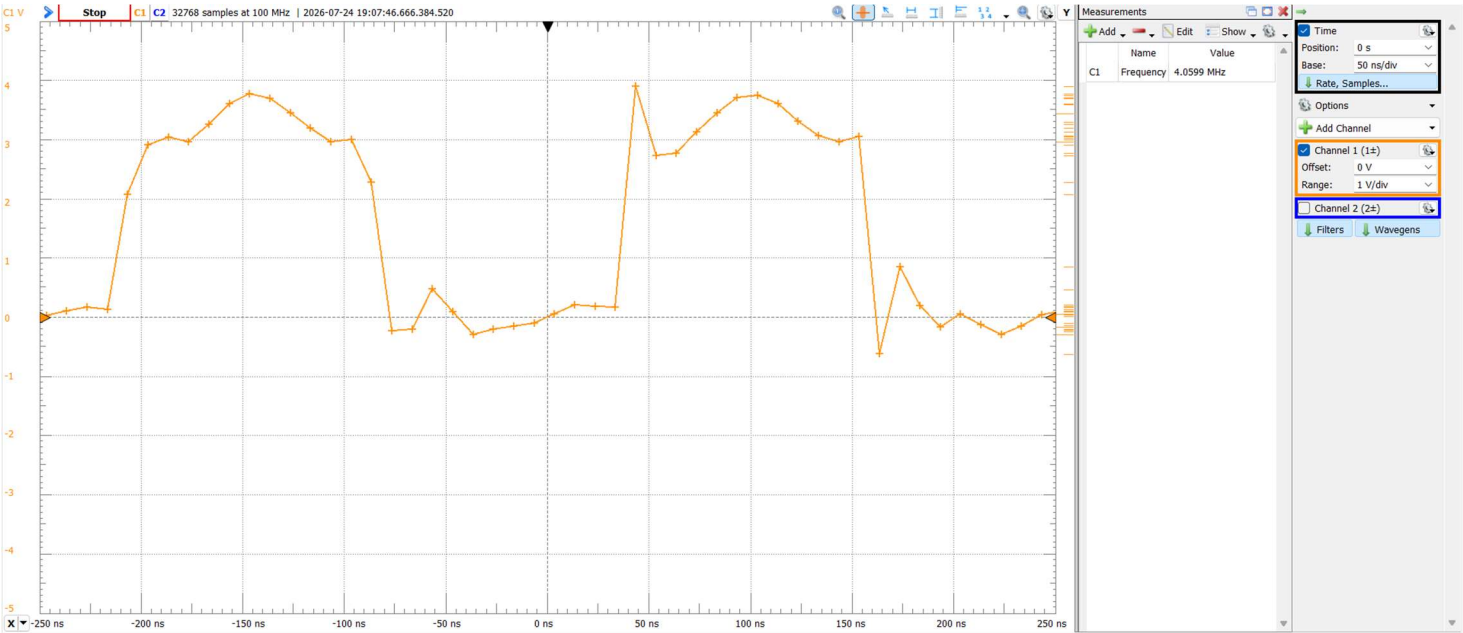


Figure 1: 4MHz Clock Frequency Scope

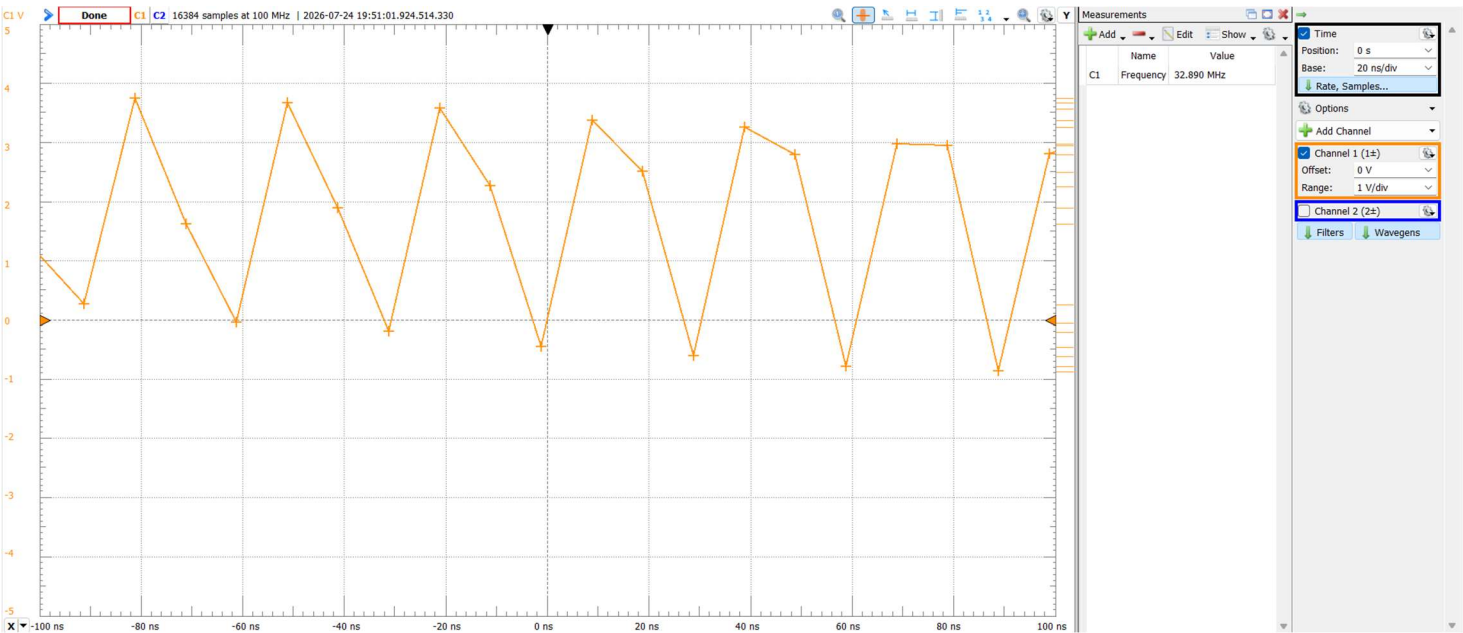


Figure 2: 32MHz Clock Frequency Scope

.s FILE

```
/* clock.s
 *
 * Last Updated: 20 May 2026
 * Authors: Wes, Dr. Schwartz
 * Configurer: Evan Baesler
 */

// GROUP CONFIGURATIONS DO NOT WORK IN .s FILES!

#include <avr/io.h>

.section .text

.global clock_init
clock_init:
    push r24

    ldi r24, OSC_RC32MEN_bm
    sts OSC_CTRL, R24           ;Enables the 32MHz internal oscillator

check32MHzStatus:
    lds r24, OSC_STATUS
    ;Ensure that the 32MHz clock is ready before proceeding
    sbrc r24, OSC_RC32MRDY_bp
    rjmp check32MHzStatus

    ;Writing to CCP disables interrupts for a certain number of cycles
    ;to give the clock time to switch sources. It also enables writes to certain registers.
    ldi r24, 0xD8
    sts CPU_CCP, r24

    ;Finally, select the now-ready 32MHz oscillator as the new clock source.
    ldi r24, 0x01
    sts CLK_CTRL, r24

skip32MHZ_enable:
    ;CPU CLK prescaler settings
    ;Use values that are powers of 2 from 1 to 512 (1, 2, 4, 8, 16, ..., 512) for A. See Table 7-2 in
the manual.
    ;You can also change B/C. See Table 7-3 in the manual.

    ldi r24, 0xD8
    sts CPU_CCP, r24

    ldi r24, ((0x05 << 2) | (0x00 << 0))
    ;ldi r24, (CLK_PSADIV_8_gc | CLK_PSBCDIV_1_1_gc) for 4MHz
    ;or ldi r24, (CLK_PSADIV_1_gc | CLK_PSBCDIV_1_1_gc) for 32 MHz

    sts CLK_PSCTRL, r24

    pop r24

    ret
```